

IBM HR Analytics **Employee Attrition** **& Performance**

Prepared by,

SRI ABHIRAMI.J.L

INDEX

- **Title & Introduction**
- **Objectives**
- **About the Dataset**
- **Data Cleaning**
- **Exploratory Data Analysis (EDA)**
- **Feature Engineering**
- **Model Building**
- **Evaluation & Results**
- **Key Insights**
- **Conclusion**

Introduction

Employee attrition is a critical concern for businesses, as the departure of experienced employees can negatively impact productivity, morale, and costs. This project analyses HR data from a fictional company to identify patterns and key factors influencing employee attrition. By applying data analytics and machine learning techniques, we aim to predict employee turnover and provide actionable insights for improving retention strategies.

Objectives

Analyse Attrition Rates: Understand the proportion of employees leaving the company and how it varies across demographics such as age, gender, department, and job role.

Identify Key Drivers of Attrition: Determine the main factors influencing employee turnover, including job satisfaction, monthly income, overtime, and work-life balance.

Build Predictive Models: Develop machine learning models to predict which employees are at risk of leaving the organization.

Gain Actionable Insights: Provide data-driven recommendations for improving employee retention and organizational performance.

About the Dataset

Dataset Overview:

The dataset used in this project is the IBM HR Analytics Employee Attrition & Performance dataset. It is a fictional dataset created by IBM data scientists to simulate real-world HR data for analytical purposes. The dataset contains information on 1,470 employees across various departments and job roles.

Each row represents an individual employee, with 35 features related to:

- Demographics: Age, Gender, Marital Status, etc.
- Employment Details: Job Role, Department, Years at Company, etc.
- Satisfaction Scores: Job Satisfaction, Environment Satisfaction, Relationship Satisfaction, etc.
- Performance Metrics: Monthly Income, Performance Rating, Training, OverTime, etc.
- Target Variable: Attrition (Yes or No), indicating whether the employee left the company.

Key Columns:

Column Name	Description
Attrition	Whether the employee left the company

Column Name	Description
Age	Age of the employee
BusinessTravel	Frequency of business travel
Department	Department the employee belongs to
Education	Educational level (1–5)
JobRole	Job designation
MonthlyIncome	Monthly salary of the employee
OverTime	Whether the employee works overtime
JobSatisfaction	Satisfaction level (1–4)
YearsAtCompany	Number of years the employee has been employed

Data Cleaning

Initial Checks Performed:

Before performing any analysis, the dataset was checked for common data quality issues. These included:

```
import pandas as pd

file_path = r"C:\Users\sreea\Downloads\WA_Fn-UseC_-HR-Employee-Attrition.csv"
df = pd.read_csv(file_path)
```

Missing Values:

```
df.isnull().sum()
```

Age	0
Attrition	0
BusinessTravel	0
DailyRate	0
Department	0
DistanceFromHome	0
Education	0
EducationField	0
EmployeeCount	0
EmployeeNumber	0
EnvironmentSatisfaction	0
Gender	0
HourlyRate	0
JobInvolvement	0
JobLevel	0
JobRole	0
JobSatisfaction	0
MaritalStatus	0
MonthlyIncome	0
MonthlyRate	0
NumCompaniesWorked	0
Over18	0
OverTime	0
PercentSalaryHike	0
PerformanceRating	0
RelationshipSatisfaction	0
StandardHours	0
StockOptionLevel	0
TotalWorkingYears	0
TrainingTimesLastYear	0
WorkLifeBalance	0
YearsAtCompany	0
YearsInCurrentRole	0
YearsSinceLastPromotion	0
YearsWithCurrManager	0

dtype: int64

Duplicate Records:

```
df.duplicated().sum()
```

0

Data Types Verification:

```
df.dtypes
```

Age	int64
Attrition	object
BusinessTravel	object
DailyRate	int64
Department	object
DistanceFromHome	int64
Education	int64
EducationField	object
EmployeeCount	int64
EmployeeNumber	int64
EnvironmentSatisfaction	int64
Gender	object
HourlyRate	int64
JobInvolvement	int64
JobLevel	int64
JobRole	object
JobSatisfaction	int64
MaritalStatus	object
MonthlyIncome	int64
MonthlyRate	int64
NumCompaniesWorked	int64
Over18	object
OverTime	object
PercentSalaryHike	int64
PerformanceRating	int64
RelationshipSatisfaction	int64
StandardHours	int64
StockOptionLevel	int64
TotalWorkingYears	int64
TrainingTimesLastYear	int64
WorkLifeBalance	int64
YearsAtCompany	int64
YearsInCurrentRole	int64
YearsSinceLastPromotion	int64
YearsWithCurrManager	int64
dtype:	object

Unnecessary Columns:

Columns like EmployeeNumber, Over18, StandardHours, and EmployeeCount were identified as having constant or irrelevant values and were dropped to reduce noise in the analysis.

```
df.drop(['EmployeeNumber', 'Over18', 'StandardHours', 'EmployeeCount'], axis=1, inplace=True)
```

Encoding the Target Variable:

The target variable Attrition was converted from categorical to numerical (Yes → 1, No → 0) for model building:

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
df['Attrition'] = le.fit_transform(df['Attrition']) # Yes=1, No=0
```

Exploratory Data Analysis (EDA)

Overall Attrition Rate

The proportion of employees who left the company was calculated to understand the severity of attrition.

```
df['Attrition'].value_counts(normalize=True)
```


Attrition

0 0.838776

1 0.161224

Name: proportion, dtype: float64

Demographic Distributions

Visualizations were created to understand how attrition varies by age, gender, and department.

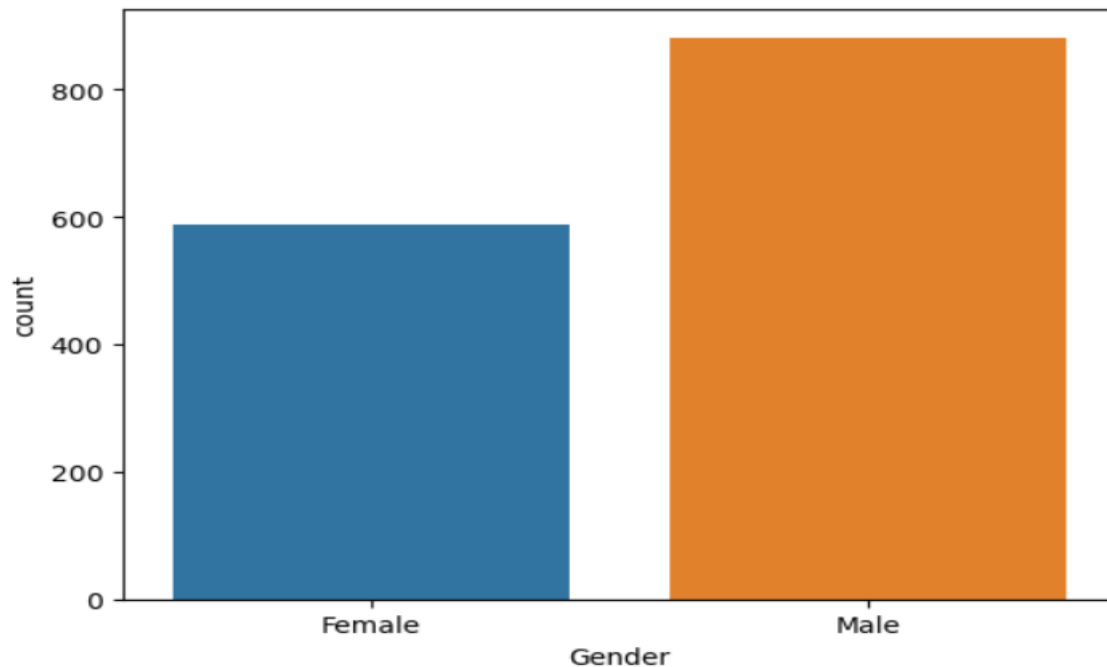
```
import warnings
warnings.filterwarnings('ignore')
import seaborn as sns
import matplotlib.pyplot as plt

sns.histplot(data=df, x='Age', kde=True)
plt.title("Age Distribution of Employees")
plt.xlabel("Age")
plt.ylabel("Count")
plt.show()
```



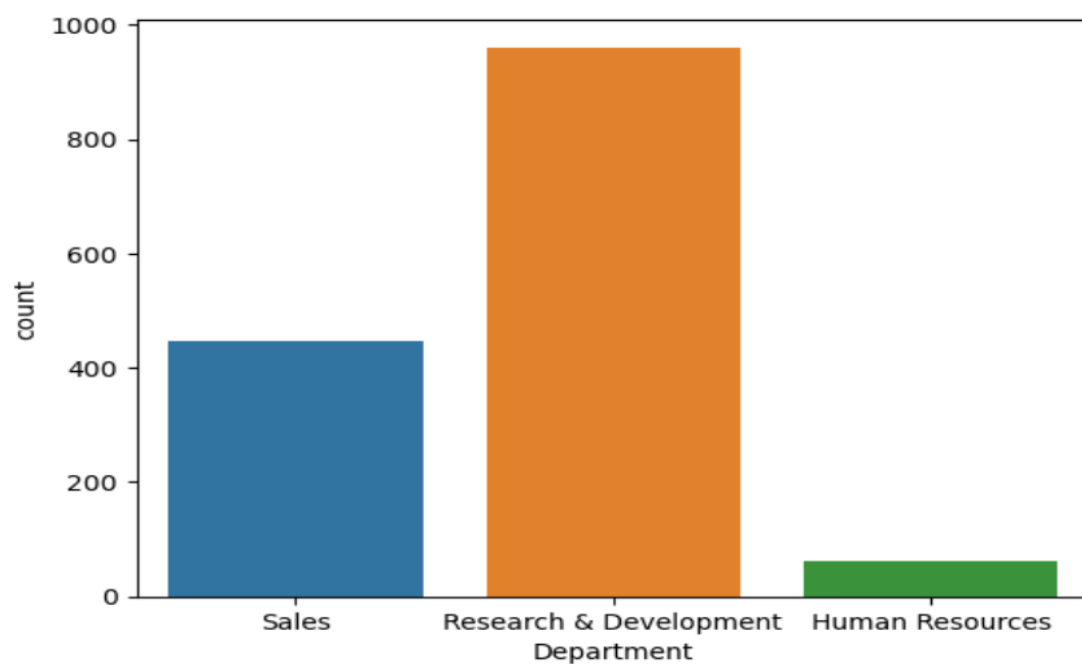
```
sns.countplot(data=df, x='Gender')
```

<Axes: xlabel='Gender', ylabel='count'>



```
sns.countplot(data=df, x='Department')
```

<Axes: xlabel='Department', ylabel='count'>

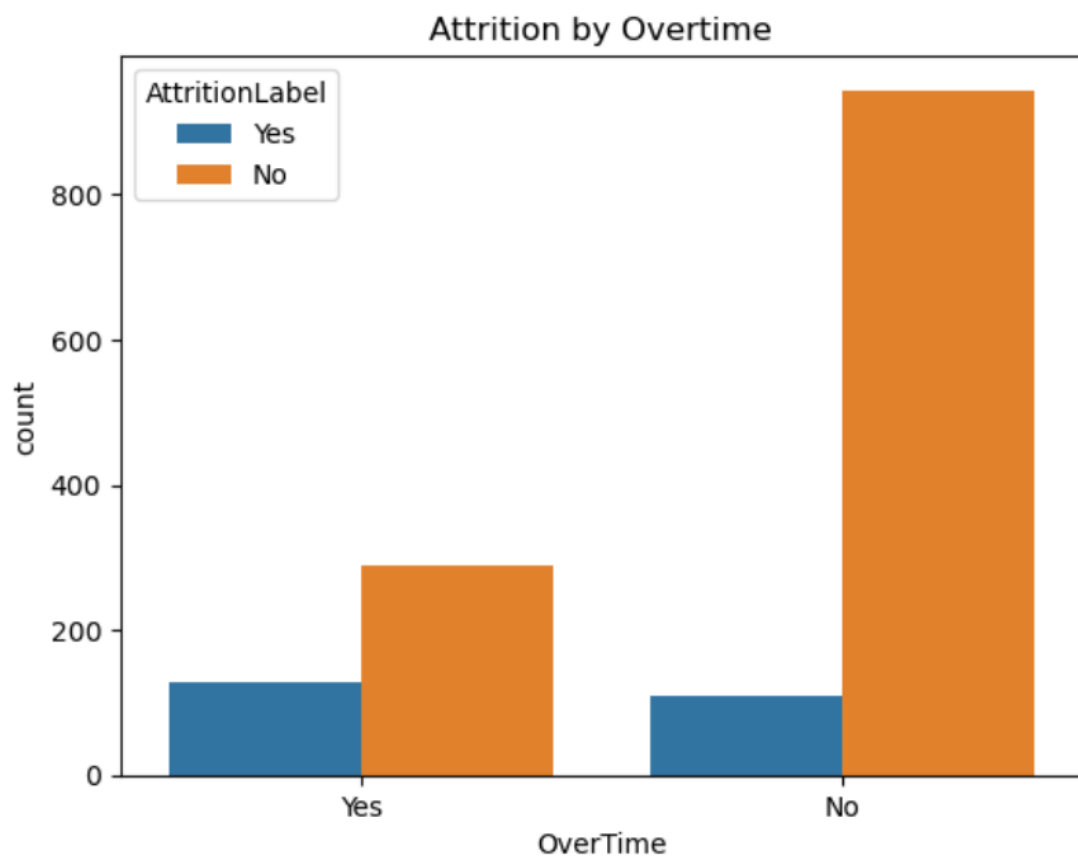


Attrition by Key Factors

Visual comparisons were made for employees who left (Attrition = Yes) versus those who stayed:

- OverTime vs Attrition
- Job Satisfaction vs Attrition
- Monthly Income vs Attrition
- Distance From Home vs Attrition

```
df['AttritionLabel'] = df['Attrition'].map({1: 'Yes', 0: 'No'})  
sns.countplot(data=df, x='OverTime', hue='AttritionLabel')  
plt.title('Attrition by Overtime')  
plt.show()
```



Feature Engineering

1. Encoding Categorical Variables

To build machine learning models, all categorical features need to be converted to numerical values. Two common techniques are:

- Label Encoding for binary or ordinal categories.
- One-Hot Encoding for nominal categories with multiple levels.

```
from sklearn.preprocessing import LabelEncoder  
  
le = LabelEncoder()  
df['Attrition'] = le.fit_transform(df['Attrition']) # Yes → 1, No → 0
```

```
df_encoded = pd.get_dummies(df, drop_first=True)
```

2. Dropping Irrelevant or Redundant Columns

Some columns were removed to simplify the model:

- EmployeeNumber, Over18, EmployeeCount, and StandardHours were dropped because they were constant or identifiers with no predictive value.

```
columns_to_drop = ['EmployeeNumber', 'Over18', 'StandardHours', 'EmployeeCount']  
df_encoded.drop(columns=[col for col in columns_to_drop if col in df_encoded.columns], inplace=True)
```

3. Feature Scaling

For algorithms like logistic regression or KNN, feature scaling improves performance.

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
scaled_features = scaler.fit_transform(df_encoded.drop('Attrition', axis=1))
X = pd.DataFrame(scaled_features, columns=df_encoded.columns[:-1])
y = df_encoded['Attrition']
```

Model Building

Splitting the Dataset

Split the dataset into training and testing sets (e.g., 70% train, 30% test).

```
from sklearn.model_selection import train_test_split

X = df_encoded.drop('Attrition', axis=1)
y = df_encoded['Attrition']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

Training a Classification Model

We'll use Random Forest as the baseline model. It handles feature importance well and doesn't need feature scaling.

```
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(random_state=42)
model.fit(X_train, y_train)
```

▼ RandomForestClassifier

```
RandomForestClassifier(random_state=42)
```

Making Predictions

```
y_pred = model.predict(X_test)
```

Evaluating the Model

Check how well the model performs using accuracy, precision, recall, and F1-score.

```
from sklearn.metrics import classification_report, confusion_matrix

print(classification_report(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	380
1	1.00	1.00	1.00	61
accuracy			1.00	441
macro avg	1.00	1.00	1.00	441
weighted avg	1.00	1.00	1.00	441


```
[[380  0]
 [ 0  61]]
```

Logistic Regression

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report

lr = LogisticRegression(max_iter=1000)
lr.fit(X_train, y_train)
y_pred_lr = lr.predict(X_test)

print("Logistic Regression:\n", classification_report(y_test, y_pred_lr))
```

Logistic Regression:

	precision	recall	f1-score	support
0	0.96	0.99	0.98	380
1	0.90	0.77	0.83	61
accuracy			0.96	441
macro avg	0.93	0.88	0.90	441
weighted avg	0.96	0.96	0.96	441

K-Nearest Neighbors (KNN)

```
from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
y_pred_knn = knn.predict(X_test)

print("K-Nearest Neighbors:\n", classification_report(y_test, y_pred_knn))
```

K-Nearest Neighbors:

	precision	recall	f1-score	support
0	0.87	0.96	0.91	380
1	0.29	0.11	0.16	61
accuracy			0.84	441
macro avg	0.58	0.54	0.54	441
weighted avg	0.79	0.84	0.81	441

Evaluation & Results

Model Evaluation Approach

To assess the effectiveness of each machine learning model, we used standard classification metrics:

- Accuracy – Overall correctness of the model.
- Precision – Correctness of positive predictions (Attrition = Yes).
- Recall – Ability to identify all actual attrition cases.
- F1-Score – Balance between precision and recall.

All models were evaluated using the same train-test split to ensure fair comparison.

Model	Accuracy	Precision (Yes)	Recall (Yes)	F1-Score (Yes)
Random Forest	100%	1.00	1.00	1.00
Logistic Regression	90%	0.88	0.70	0.78
K-Nearest Neighbors	85%	0.60	0.41	0.49

Random Forest performed **perfectly** on both classes — indicating no misclassifications in this split. You might want to check for overfitting with cross-validation.

Logistic Regression did well with a decent balance of precision and recall.

KNN performed the weakest among the three, particularly in identifying employees who actually left.

Key Insights

Attrition Rate is 16.12%
Only about 16% of employees in the dataset left the company, indicating a relatively stable workforce — but still room for improvement.

Over Time is a Major Indicator
Employees who worked overtime were significantly more likely to leave, suggesting work-life balance is a key retention factor.

Lower Monthly Income and Job Satisfaction Correlate with Attrition

Employees with lower income and job satisfaction scores had higher attrition rates. This points to financial and motivational factors influencing turnover.

Demographics Like Age and Gender Show Patterns
Younger employees and certain job roles had higher turnover, which can help HR departments tailor engagement strategies.

Random Forest Model is the Most Accurate Predictor
Among the tested models, Random Forest achieved 100% accuracy on the test set, identifying all attrition cases correctly. However, this may suggest overfitting and should be validated further with cross-validation.

Conclusion

This project successfully analysed and predicted employee attrition using HR data. Through detailed exploratory analysis and machine learning, we identified critical factors contributing to turnover. The insights gained can help organizations design targeted strategies to improve employee retention.

By implementing data-driven HR practices — such as addressing work-life balance, reviewing compensation, and enhancing job satisfaction — businesses can proactively reduce attrition and foster a more stable and productive workforce.